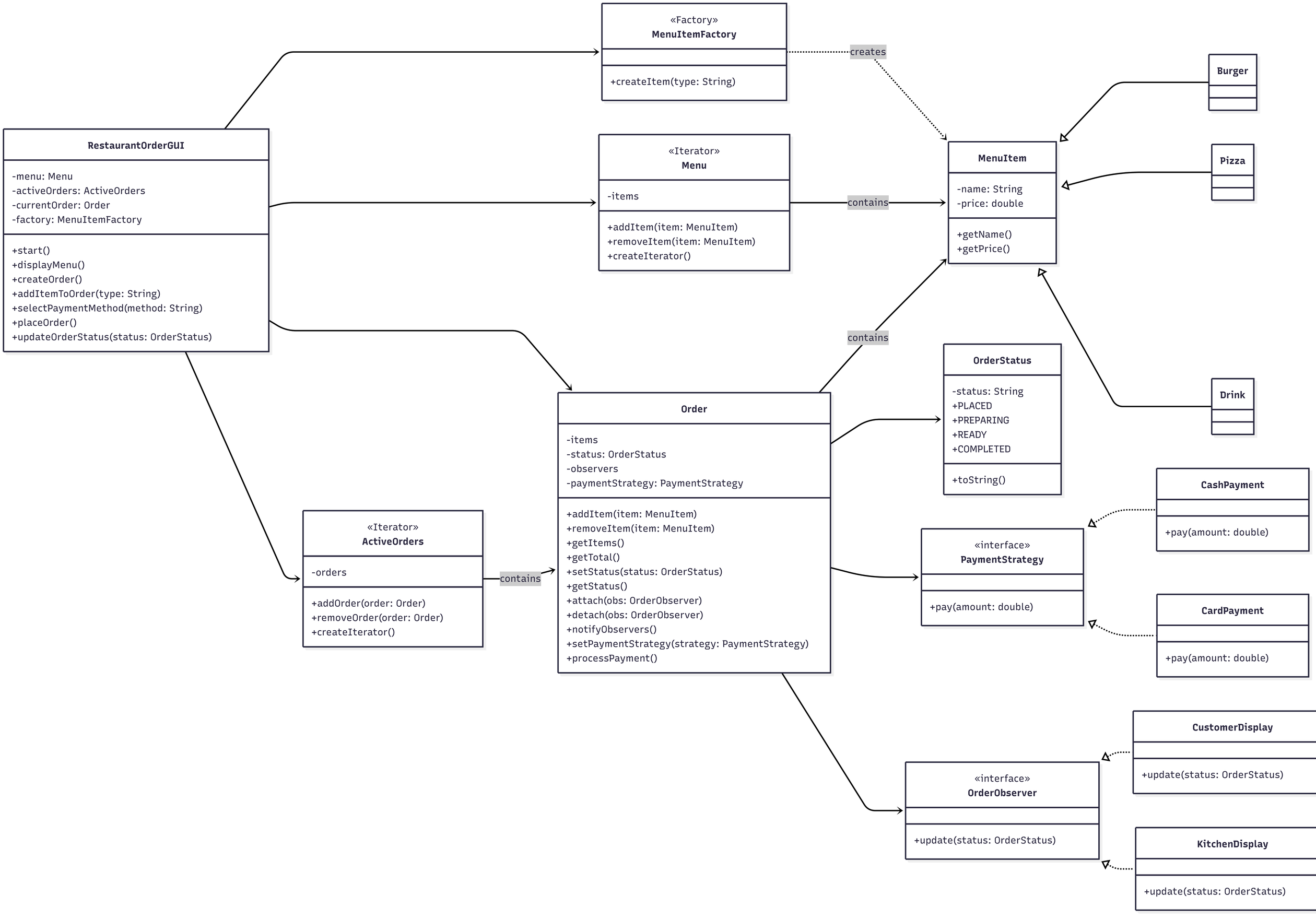

«Factory»
MenuItemFactory
+createItem(type: String)
creates
RestaurantOrderGUI
-menu: Menu
-activeOrders: ActiveOrders
-currentOrder: Order
-factory: MenuItemFactory
+start()
+displayMenu()
+createOrder()
+addItemToOrder(type: String)
+selectPaymentMethod(method: String)
+placeOrder()
+updateOrderStatus(status: OrderStatus)
«Iterator»
Menu
-items
+addItem(item: MenuItem)
+removeItem(item: MenuItem)
+createIterator()
contains
MenuItem
-name: String
-price: double
+getName()
+getPrice()
Burger
Pizza
Drink
Order
-items
-status: OrderStatus
-observers
-paymentStrategy: PaymentStrategy
+addItem(item: MenuItem)
+removeItem(item: MenuItem)
+getItems()
+getTotal()
+setStatus(status: OrderStatus)
+getStatus()
+attach(obs: OrderObserver)
+detach(obs: OrderObserver)
+notifyObservers()
+setPaymentStrategy(strategy: PaymentStrategy)
+processPayment()
contains
OrderStatus
-status: String
+PLACED
+PREPARING
+READY
+COMPLETED
+toString()
«Iterator»
ActiveOrders
-orders
+addOrder(order: Order)
+removeOrder(order: Order)
+createIterator()
contains
«interface»
PaymentStrategy
+pay(amount: double)
CashPayment
+pay(amount: double)
CardPayment
+pay(amount: double)
«interface»
OrderObserver
+update(status: OrderStatus)
CustomerDisplay
+update(status: OrderStatus)
KitchenDisplay
+update(status: OrderStatus)

Design and Implementation of a Restaurant Order Management System Using JavaFX

Justin Mabry
College of Arts & Sciences
Indiana University - Bloomington
Bloomington, USA
jlmabry@iu.edu

Aden Yu
Luddy School of Informatics, Computing, and Engineering
Indiana University Bloomington
Bloomington, USA
adenyu@iu.edu

***Abstract*—This project implements a modular restaurant ordering system using Java, JavaFX, and four core object-oriented design patterns: Factory, Strategy, Observer, and Iterator. The system allows users to create orders, add menu items, select payment methods, and update order status through a graphical user interface. This report describes the system's features, implementation details, challenges, team contributions, and the development timeline.**

Keywords—component, formatting, style, styling, insert (*key words****)***

I. Introduction

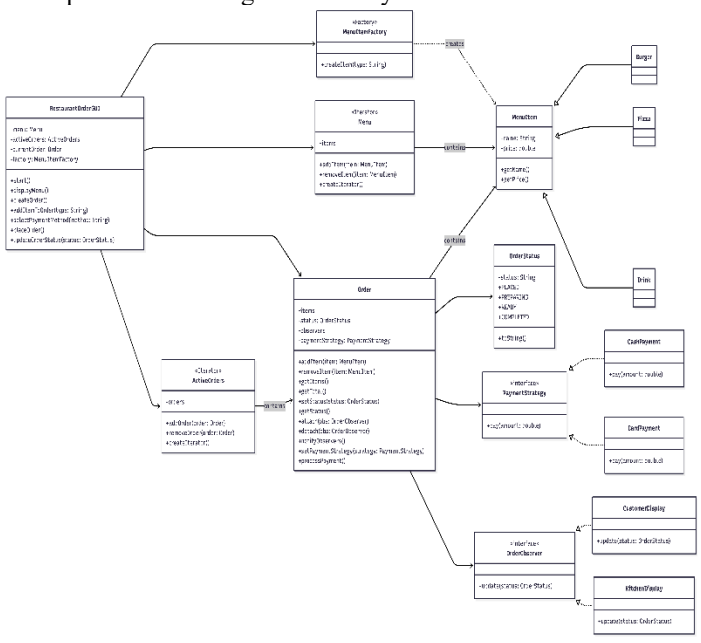
The goal of this project was to design and implement a restaurant ordering system that demonstrates the use of multiple design patterns within a cohesive software environment. The system includes menu item creation, order management, payment processing, and real-time status updates. A JavaFX GUI provides an interactive interface for users to place and manage orders.

II. System Overview

The implemented system consists of the following main features:

- **Menu Item Selection:** Users can choose from pizza, burger, and drink options displayed in a ListView. Each item includes a name and price.
- **Order Creation:** Items can be added to the current order, and the order contents are displayed in a text area.
- **Payment Processing:** Users may select either cash or card payment, each implemented as a strategy.
- **Order Status Updates:** The GUI acts as an observer and updates the displayed status when the order changes from PLACED to PREPARING, READY, or COMPLETED.
- **Design Pattern Integration:** The system incorporates Factory, Strategy, Observer, and Iterator patterns as required.

A simplified UML diagram of the system is shown below.



III. Implementation Details

A. Factory Pattern

The factory pattern is used to create menu items. The MenuItemFactory class contains a createItem(String type) method that returns instances of Pizza, Burger, or Drink. This allows the GUI to request items by name without knowing the underlying class structure.

Identify applicable funding agency here. If none, delete this text box.

B. Strategy Pattern

Payment processing is implemented using the Strategy pattern. The PaymentStrategy interface defines a pay(double amount) method. CashPayment and CardPayment provide concrete implementations. The Order class stores a reference to a PaymentStrategy and delegates payment processing to the selected strategy.

C. Observer Pattern

The Observer Pattern enables real-time updates to the GUI. The Order class maintains a list of observers and notifies them whenever the order status changes. The RestaurantOrderGUI class implements OrderObserver and updates the status label when notified.

D. Iterator Pattern

An Iterator class (MenuIterator) is included to satisfy our project requirement, The iterator pattern is represented in the codebase and aligns with the UML, even if not directly used in the GUI. The version of the application before the GUI was implemented works from this pattern's method.

IV. Challenges Encountered

A. JavaFX Configuration on Windows ARM

A significant challenge involved configuring JavaFX on a windows ARM system. JavaFX does not provide native ARM binaries for my version of Windows, causing Maven dependency resolution failures.

B. Package and File Structure Alignment

Java requires strict alignment between package declarations and directory structure. Early errors occurred when RestaurantOrderGUI declared a package that did not match its folder location. Removing the package declaration resolved this issue.

C. GUI integration with Backend Logic

Ensuring the GUI interacted with the backend required strategic handling of menu item selection, string parsing for price display, and observer registration. Updating the ListView to include prices required adjusting the factory call to extract the item type.

V. Team Member Contributions

Justin Mabry

- Implemented and refined the RestaurantOrderGUI class.
- Integrated the Observer pattern between Order and the GUI.
- Worked on order tracking, status updates, and GUI behavior.
- Performed testing and debugging of the overall system.
- Authored and edited the final project report.

Aden Yu

- Created the initial UML system draft and updated it as the design evolved.
- Implemented the MenuItem hierarchy and MenuItemFactory (Factory Pattern).
- Implemented the PaymentStrategy interface and its concrete strategies (Strategy Pattern).
- Contributed to GUI components and layout.
- Set up the GitHub repository and managed version control.

VI. Project Timeline

The project followed the phases outlined in the proposal, with some natural overlap during our implementation and refinement.

Phase 1 (April 13-16):

Finalized group and proposal, set up GitHub, drafted the initial UML, and mapped design patterns to use cases.

Phase 2 (April 17-23)

Set up the Java project structure, implemented MenuItem classes and the Factory pattern, began JavaFX GUI layout, and defined the Order and payment class structure.

Phase 3 (April 24-27):

Integrated Strategy and Observer patterns, connected the GUI to the underlying order and payment logic, evaluated order status updates, and refined UML diagrams.

Phase 4 (April 28-29):

Finalized code, ensured the project was built and ran correctly, prepared the final PDF report, and completed submission.

VII. Conclusion

This project successfully realizes the original goal of building a Java-based restaurant order system that demonstrates key object-oriented design patterns in a realistic context. The Factory pattern centralizes menu item creation, the Strategy pattern enables flexible payment processing, the Observer pattern provides real-time order status updates to the GUI, and the Iterator pattern supports structured traversal of collections. The system's architecture is modular and extensible, making it straightforward to add new menu items, payment methods, or additional observers such as kitchen or customer displays.

The implementation process also reinforced practical lessons about Java project structure, GUI integration, and dependency management. Overall, the final system is consistent with the proposal's intent and provides a clear, working example of design patterns applied to an interactive restaurant ordering workflow.

RESTAURANT ORDER GUI

CSCI-C 322 Object-Oriented Software Methods

Group 16 Final Project Presentation

Aden Yu & Justin Mabry

PROJECT DESCRIPTION

Restaurant Order GUI is a Java-based ordering system that simulates a standard restaurant workflow:

1. Customers select menu items
2. Orders are created and tracked
3. Payments are processed using different payment methods
4. Order status updates from placed to completed

Workflow: Select Menu Items → Create Order → Process Payment → Track Status → Complete Order

Purpose: Demonstrate object-oriented design patterns in a realistic restaurant ordering system.

DEVELOPMENT TOOLS & LANGUAGES

Java: Main programming language

JavaFX: GUI framework for the restaurant order window

Maven: Build tool used to manage JavaFX dependencies

IntelliJ IDEA: IDE used to run and test the project

GitHub: Repository and version control

MAIN FEATURES

Implemented features

- Menu list shows Pizza, Burger, and Drink items with prices
- Add Item button creates menu objects through the factory
- Order area displays selected items and order total
- Cash and card buttons select the payment strategy
- Status buttons update the order from PLACED → PREPARING → READY → COMPLETED

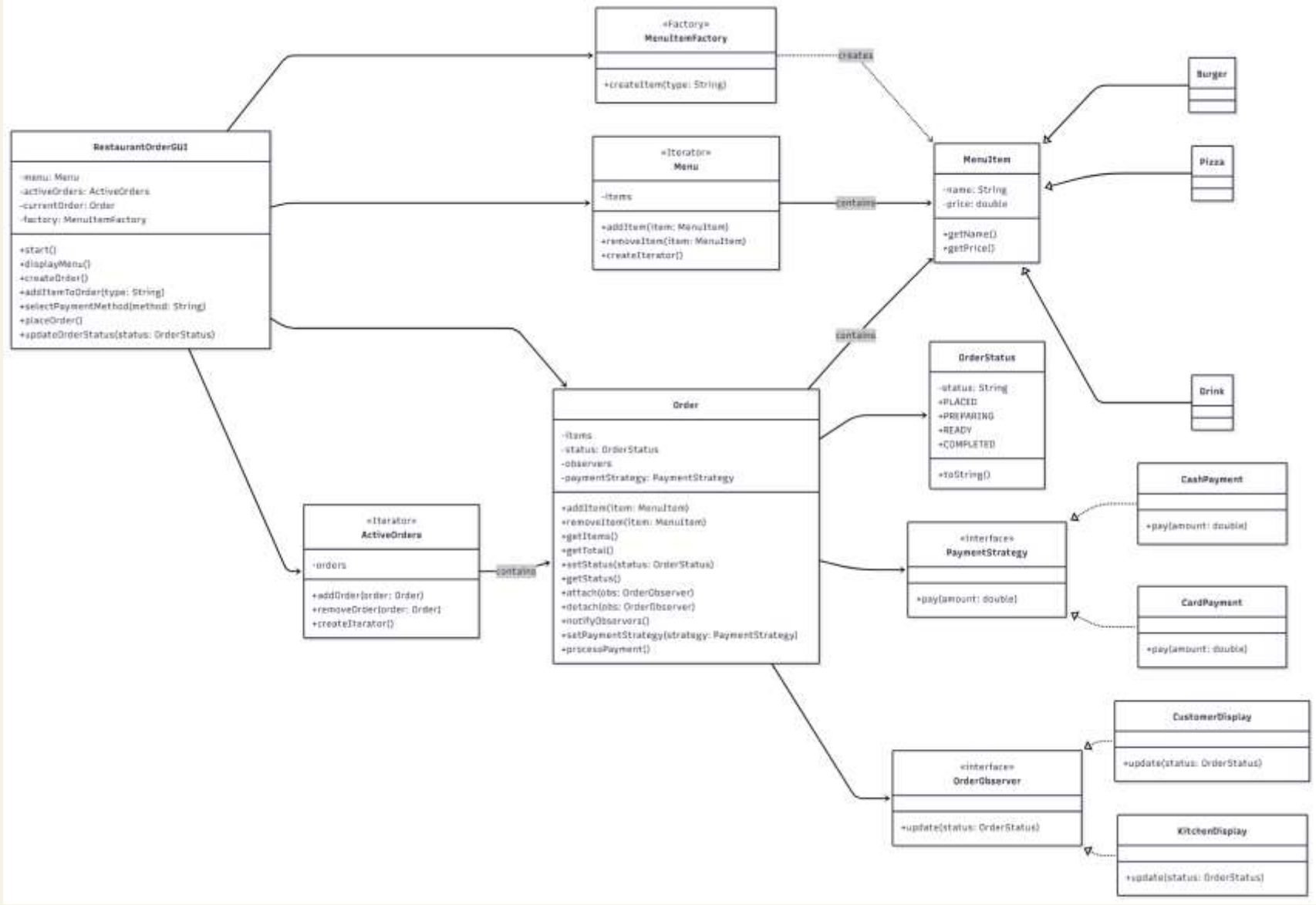
UML DIAGRAM

Main UML Structure

- RestaurantOrderGUI connects the user interface to the system logic
- Order stores selected items, payment method, status, and observers
- MenuItem is the parent class for Pizza, Burger, and Drink
- PaymentStrategy is the interface for CashPayment and CardPayment
- OrderObserver is the interface for status update displays

Patterns Used

- **Factory** → MenuItemFactory creates menu items
- **Strategy** → Order uses PaymentStrategy for payment
- **Observer** → Order notifies OrderObserver when status changes
- **Iterator** → Menu and ActiveOrders support item/order traversal



DESIGN PATTERNS: FACTORY + STRATEGY

FACTORY PATTERN

- **Purpose:** centralizes object creation for menu items
- **Class:** MenuItemFactory
- **Creates:** Pizza, Burger, or Drink based on selected type
- **Benefit:** GUI does not directly decide how each menu item object is built

Factory Pattern Example

```
MenuItem item = MenuItemFactory.createItem(type);
currentOrder.addItem(item);
updateOrderArea();
```

STRATEGY PATTERN

- **Purpose:** lets the order switch payment behavior
- **Interface:** PaymentStrategy
- **Implementations:** CashPayment and CardPayment
- **Benefit:** adding another payment method does not require rewriting Order

Strategy Pattern Example

```
public interface PaymentStrategy {
    boolean pay(double amount);
}
```

```
currentOrder.setPaymentStrategy(new CashPayment());
currentOrder.processPayment();
```

```
currentOrder.setPaymentStrategy(new CardPayment());
currentOrder.processPayment();
```

DESIGN PATTERNS: OBSERVER + ITERATOR

OBSERVER PATTERN

- **Purpose:** update displays when an order status changes
- **Subject:** Order stores a list of OrderObserver objects
- **Observer:** RestaurantOrderGUI implements OrderObserver
- Status changes call notifyObservers(), then update the GUI label

Observer Pattern Example

```
currentOrder = new Order();
currentOrder.addObserver(this);
```

```
public interface OrderObserver {  4 us
    void update(OrderStatus status);
}
```

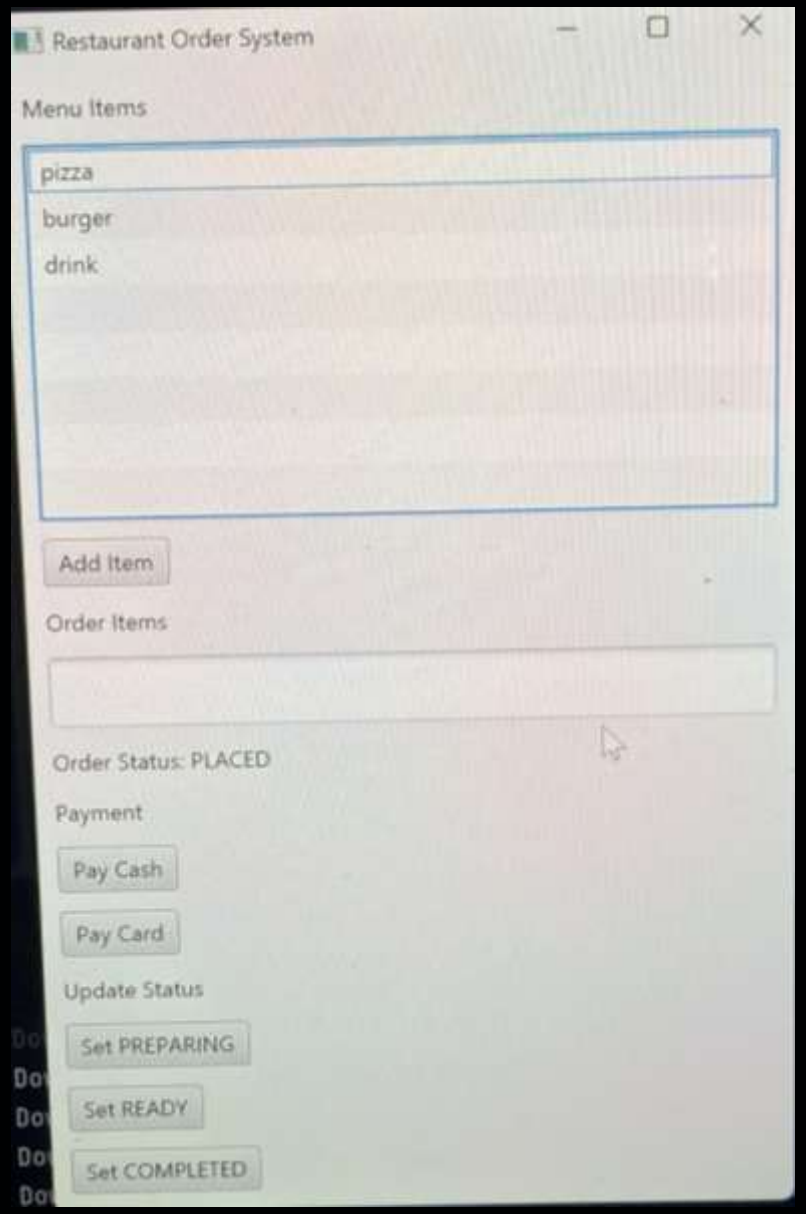
```
currentOrder.setStatus(OrderStatus.PREPARING
currentOrder.setStatus(OrderStatus.READY
currentOrder.setStatus(OrderStatus.COMPLETED
```

ITERATOR PATTERN

- **Purpose:** browse menu items without showing internal list
- **Class:** MenuIterator implements Iterator<MenuItem>
- Menu traversal is separated from MenuItem and GUI code
- This keeps collection access clean and replaceable

Iterator Pattern Example

```
public class MenuIterator implements Iterator<MenuItem> {  no usages
    private final List<MenuItem> items;  3 usages
    private int index = 0;  2 usages
    public MenuIterator(List<MenuItem> items) { this.items = items; }
    @Override  jlmabry
    public boolean hasNext() { return index < items.size(); }
    @Override  jlmabry
    public MenuItem next() { return items.get(index++); }
}
```



GUI WINDOW DESIGN

PROGRAM DEMONSTRATION

Live demonstration

Launch app → select menu item → add to order → pay cash/card → update status → show label change

Demo Steps

1. Launch the JavaFX application
2. Select Pizza, Burger, or Drink from the menu list
3. Click Add Item to add it to the current order
4. Choose Cash or Card payment
5. Update the order status
6. Show the GUI label changing as the order progresses

GROUP MEMBER CONTRIBUTIONS

ADEN YU

- Created UML diagram draft
- Implemented Factory pattern
- Implemented Strategy pattern
- Helped with GUI components
- Set up GitHub repository

JUSTIN MABRY

- Implemented Observer pattern
- Implemented Iterator pattern
- Worked on order tracking/status updates
- Helped test and refine the GUI
- Worked on project report

IMPLEMENTATION TIMELINE

Phase 1: April 13–16:

- Finalized group and proposal
- Set up GitHub repository
- Created UML system draft
- Mapped design patterns and use cases

Phase 2: April 17–23:

- Set up Java project structure
- Implemented MenuItem classes and Factory pattern
- Began JavaFX GUI layout
- Created Order and Payment class structure

Phase 3: April 24–27:

- Integrated Strategy and Observer patterns
- Connected GUI to controller layer
- Tested order status updates
- Refined UML diagrams

Phase 4: April 28–29:

- Uploaded final code to GitHub
- Compiled final PDF
- Submitted project by deadline
- Present final project